Gérer les Formulaires en React : Inputs, Checkbox et Boutons Radios

PAR L'ÉQUIPE ALPHORM 18 JANVIER 2025

Cet article fait partie du guide **Tutoriel React JS : Apprenez à Créer des Applications Modernes avec des Projets Concrets**, partie 11 sur 13.

Gérer plusieurs contrôles de formulaire en ReactJS peut rapidement devenir complexe, entraînant des bugs et des comportements inattendus.

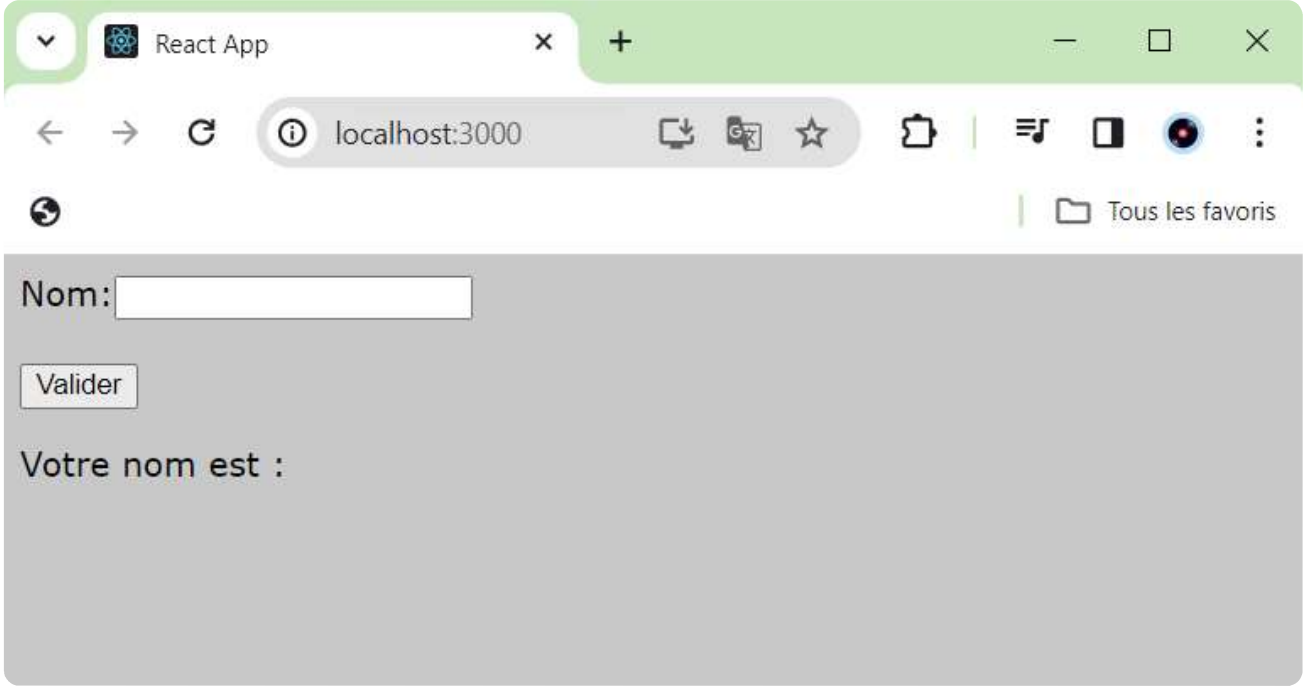
Ces complications augmentent la frustration des développeurs et affectent l'expérience utilisateur. Un formulaire mal géré peut nuire à la performance de l'application.

Cet article vous guide sur la gestion des inputs, checkboxes, radios, et autres éléments en ReactJS. Découvrez comment simplifier la gestion des états de vos composants pour des formulaires plus fluides et des applications robustes.

Contrôler un input en ReactJS

Donc , pour commencer sur cette partie, nous allons commencer par le contrôle d'un input.

Contrôler un input en React consiste à gérer et à synchroniser l'état de l'input avec celui de l'application. Plutôt que de laisser le DOM gérer l'état de l'input, React propose d'utiliser un état local (**useState**) pour stocker la valeur de l'input. Cela permet à l'application de garder une trace de ce qui est saisi dans l'input et de réagir en conséquence aux changements. En contrôlant l'input, on peut effectuer des validations en temps réel, des transformations ou des actions en réponse aux modifications de la valeur saisie. Pour vous permettre d'approfondir votre compréhension des concepts que nous avons abordés, je vais illustrer cela avec l'image ci-dessous.



D'accord, nous avons déjà un champ de saisie, et la valeur de cet input sera immédiatement affichée en dessous. Je suppose que vous avez une idée de comment procéder, mais nous allons le faire ensemble.

Pour le faire, rendez-vous dans le dossier **src/component/formulaire.js**

```
import React, { useState } from "react";
export default function Formulaire(){
const [valueName, setValueName] = useState('Elon Musk');
const changer = (e) => {
setValueName(e.target.value);
}
return(
<>
<form>
<label htmlFor="nom">Nom:</label>
<input
type="text"
id="nom"
value={valueName}
onChange={changer}
/><br/><br/>
<input
```

```
type="submit"
value= 'Valider'
onClick={(e)=>{
e.preventDefault();
}}
/>
</form>
<div>
<p>Votre nom est : {valueName}</p>
</div>
</>
);
}
```

- **import React, { useState } from « react »:** Cette ligne importe les fonctionnalités nécessaires depuis la bibliothèque React, notamment la fonction **useState** pour gérer l'état local des composants.
- **const [valueName, setValueName] = useState('Elon Musk'):** Ici, nous utilisons le hook d'état **useState** pour déclarer une variable d'état appelée **valueName**, qui sera utilisée pour stocker la valeur de l'input. La valeur initiale est définie sur 'Elon Musk', mais elle peut être modifiée par la suite.
- **const changer = (e) => { setValueName(e.target.value); }:** Cette fonction changer est définie pour mettre à jour l'état **valueName** en fonction de la valeur saisie dans l'input. Elle est appelée à chaque fois que l'utilisateur modifie le contenu de l'input.

Dans la fonction de rendu, un formulaire est rendu avec un champ de saisie pour le nom et un bouton de validation. L'attribut value de l'input est lié à **valueName**, donc la valeur de l'input sera toujours synchronisée avec **valueName**.

Lorsque l'utilisateur saisit du texte dans l'input, la fonction changer est déclenchée par l'événement onChange, mettant à jour l'état valueName avec la nouvelle valeur saisie.

Le bouton de validation a un gestionnaire d'événements onClick qui empêche le comportement par défaut du formulaire à l'aide de e.preventDefault(). Cela empêche la page de se recharger lorsque le bouton est cliqué, ce qui est souvent le comportement par défaut d'un bouton dans un formulaire.

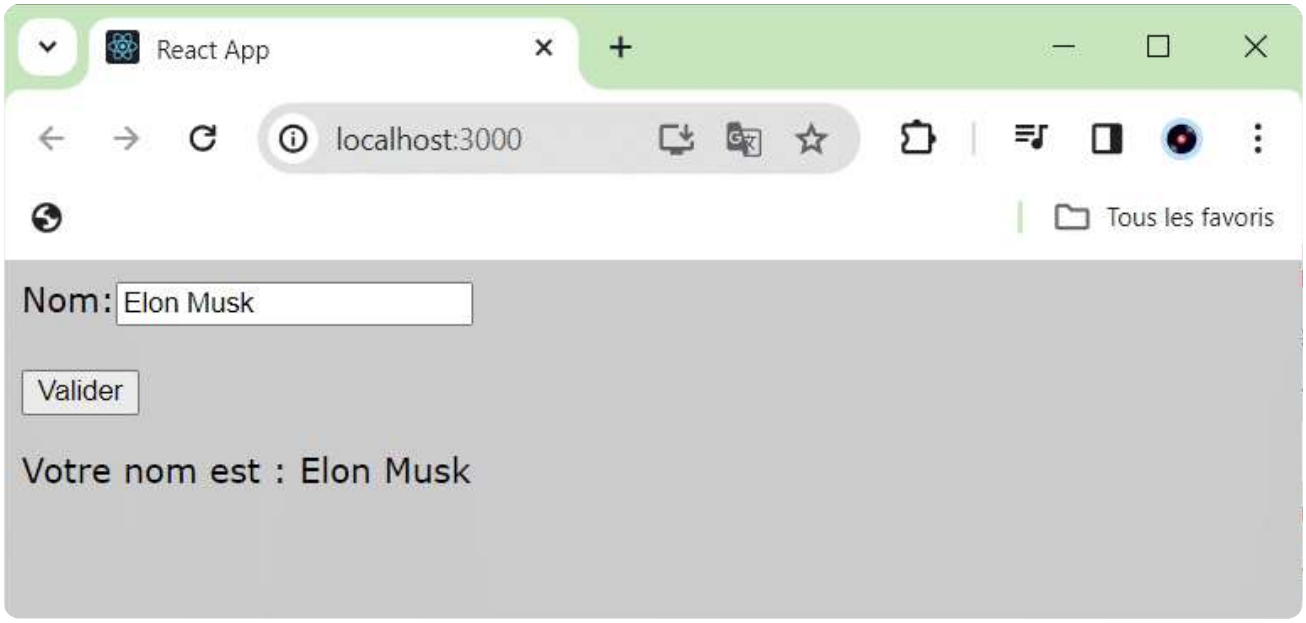
Et pour que le composant soit fonctionnel. N'oubliez pas l'importer dans le fichier **index.js**

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './styles/index.css'
```

```
import Formulaire from './component/formulaire';
const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
<React.StrictMode>
<Formulaire/>
</React.StrictMode>
);
```

Nous venons d'importer notre composant, puis nous l'avons appelé en utilisant la balise <Formulaire>.

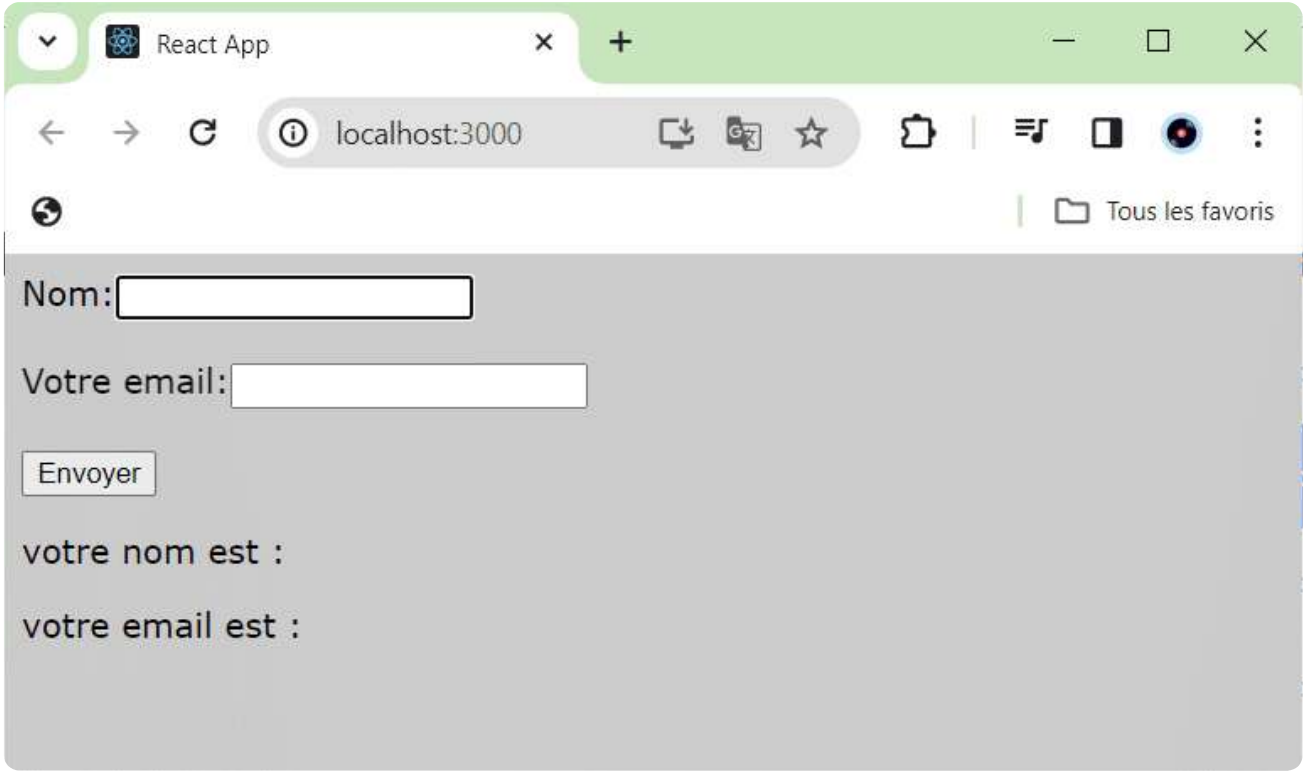
En démarrant votre projet, cela va nous donner quelque chose comme ça :



Vous avez la liberté de saisir le nom de votre choix, et celui-ci sera toujours affiché dans la boîte située en dessous.

Contrôler plusieurs inputs en ReactJS

Le principe demeure similaire à ce que nous venons d'expliquer, mais cette fois-ci, il concerne la gestion de plusieurs champs de saisie comme ceci :



Donc pour le faire, examinons la modification au niveau de notre composant Formulaire : **src/component/formulaire.js.**

```
import { useState } from "react";
export default function Formulaire(){
const [formData,setFormData] = useState(
{
nom:'Elon musk',
email:'elon@gmail.com',
}
);
const changer = (e)=>{
let name = e.target.name
let value = e.target.value
setFormData((datas) =>({
...datas ,
[name]:value
}))
}
return(
<>
<form  >
<label>Nom:</label>
```

```
<input
type="text"
id="nom"
name = 'nom'
value={formData.nom}
onChange={changer}
></input><br/><br/>
<label>Votre email:</label>
<input
type="text"
id="email"
name = 'email'
value={formData.email}
onChange={changer}
></input><br/><br/>
<input
type="submit"
value= 'Envoyer'
onClick={(e)=>{
e.preventDefault()
}}
></input>
</form>
<div>
<p>votre nom est : {formData.nom}</p>
<p>votre email est : {formData.email}</p>
</div>
</>
)
}
```

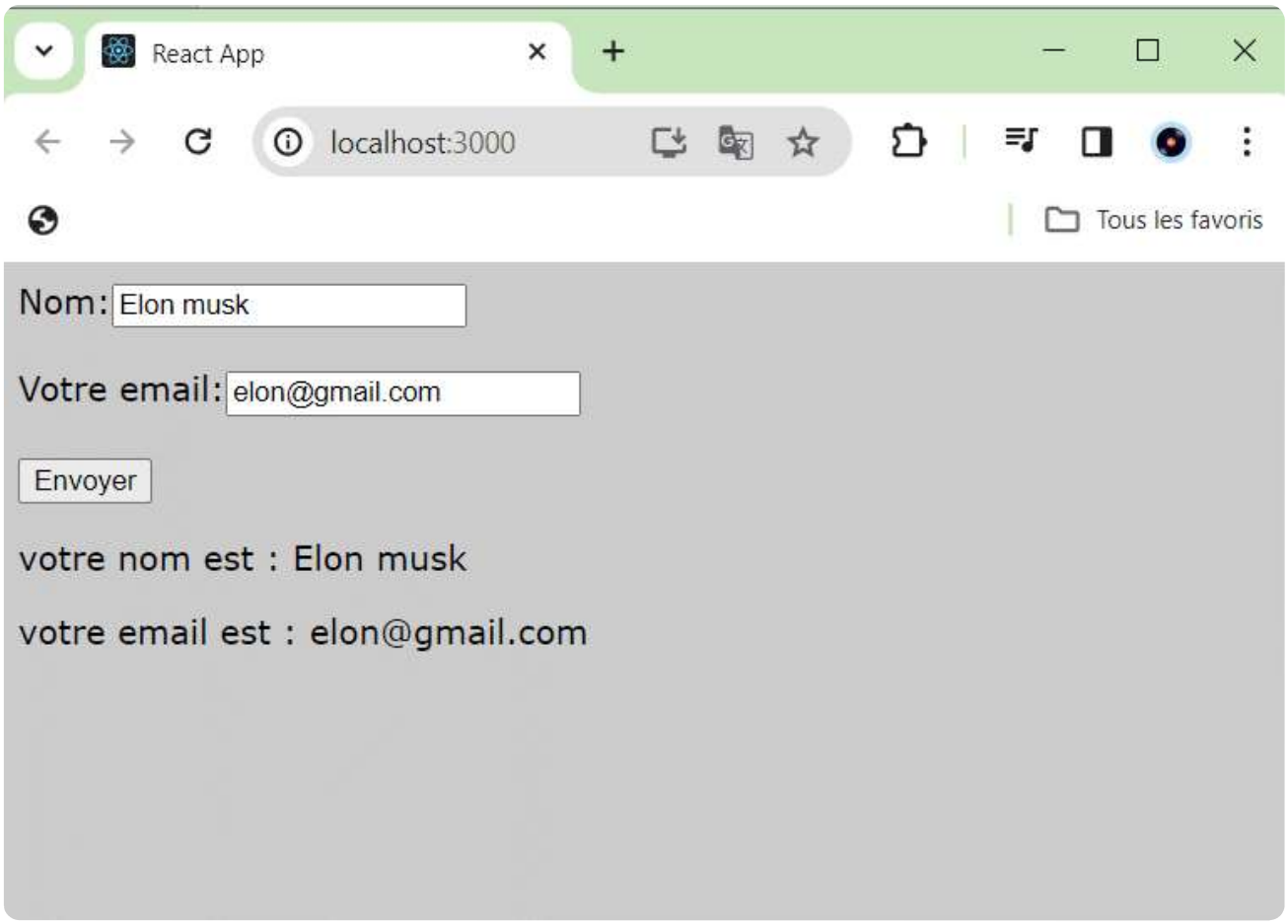
`const [formData, setFormData] = useState({nom:'Elon Musk', email:'elon@gmail.com'}):` Ici, nous utilisons le hook d'état **useState** pour déclarer une variable d'état appelée **formData**, qui contient **les données** du formulaire. Au départ, les valeurs initiales pour les champs nom et email sont définies.

La fonction changer est définie pour mettre à jour l'état **formData** (comme on a déjà expliqué dans le chapitre précédent) lorsqu'un champ de saisie est modifié. Elle récupère le nom et la valeur du champ de saisie modifié, puis utilise la fonction **setFormData** pour mettre à jour l'état en fusionnant les nouvelles données avec les données existantes à l'aide de la syntaxe de décomposition (…datas) et en remplaçant la valeur du champ modifié.

Dans le rendu du composant, un formulaire est rendu avec deux champs de saisie : un pour le nom et un pour l'e-mail. Les valeurs de ces champs sont liées à **formData.nom** et **formData.email**, respectivement, afin que les valeurs des champs soient toujours synchronisées avec l'état **formData**.

Lorsque l'utilisateur modifie un champ de saisie, la fonction changer est déclenchée par l'événement onChange, ce qui met à jour l'état **formData** avec les nouvelles valeurs saisies.

Cet extrait de code nous donne comme ceci en lançant notre projet :



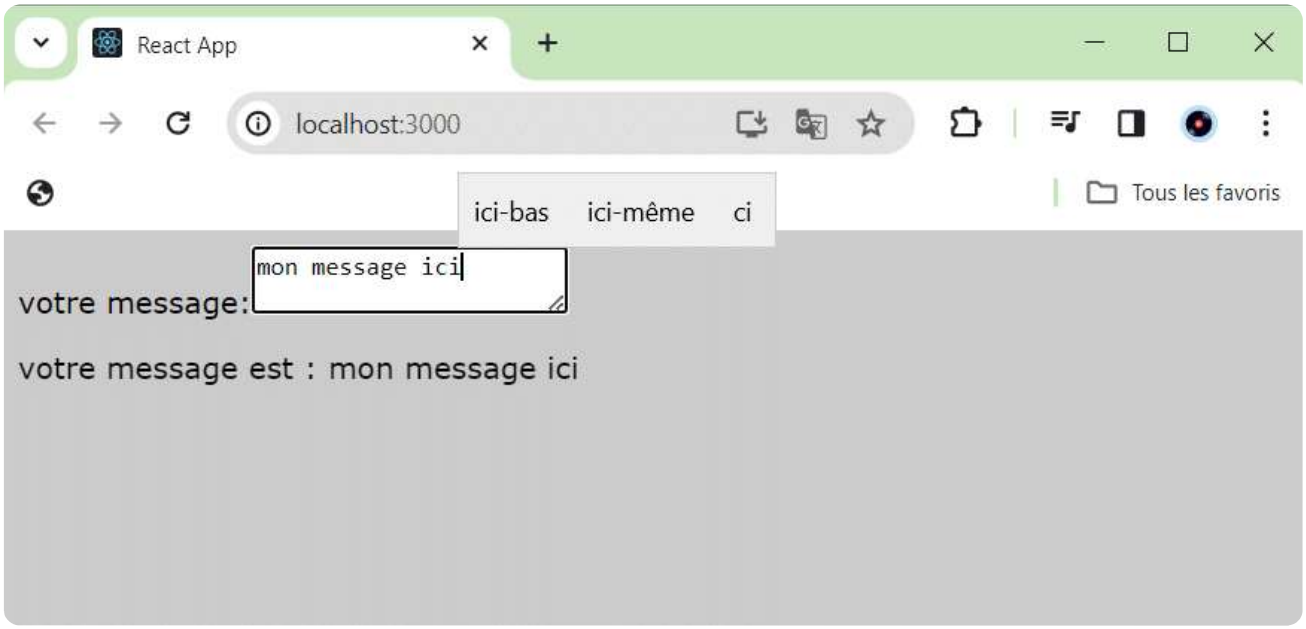
Voilà, parfait on a obtenu ce que nous avons attendu.

Contrôler un textarea en ReactJS

Dans cette nous aborderons la manière de contrôler un textarea en ReactJS. Contrôler un textarea signifie que sa valeur est gérée par React via l'état du composant plutôt que de laisser le DOM gérer sa propre valeur. Cela permet un contrôle plus précis et facilite la manipulation et la validation des données utilisateur.

Nous explorerons les méthodes pour réaliser cela efficacement tout en maintenant la réactivité et la cohérence de l'interface utilisateur.

Imaginons dans votre projet, on vous demande coder un formulaire pour le textarea comme ceci en reactjs :



Vous semblez trouver cela étrange, car il a spécifiquement mentionné l'utilisation de ReactJS plutôt que **JavaScript** seul. Ne vous inquiétez pas, vous allez bientôt le maîtriser. Maintenant, examinons le composant pour voir comment l'implémenter : **src/component/formulaire.js**

```
import { useState } from "react";
export default function Formulaire(){
const [formmsg,setFormmsg] = useState('Bonjour');
const changer = (e)=>{
setFormmsg(e.target.value);
}
return(
<>
<form  >
<label>votre message:</label>
<textarea
id="msg"
name="msg"
value={formmsg}
onChange={changer}
></textarea>
</form>
<div>
```

```
<p>votre message est : {formmsg}</p>
</div>
</>
)
}
```

Comme vous comprenez déjà, **import { useState } from « react »:** Cette ligne importe la fonction **useState** depuis la bibliothèque React, qui est utilisée pour déclarer un état local dans un composant fonctionnel.

- **const [formmsg, setFormmsg] = useState('Bonjour'):** Ici, un état local est déclaré en utilisant **useState.**
- **formmsg** est initialisé avec la valeur 'Bonjour' et **setFormmsg** est une fonction qui sera utilisée pour mettre à jour la valeur de **formmsg**.

Dans la partie du rendu (return(…)), un formulaire contenant un champ de texte (textarea) est rendu.

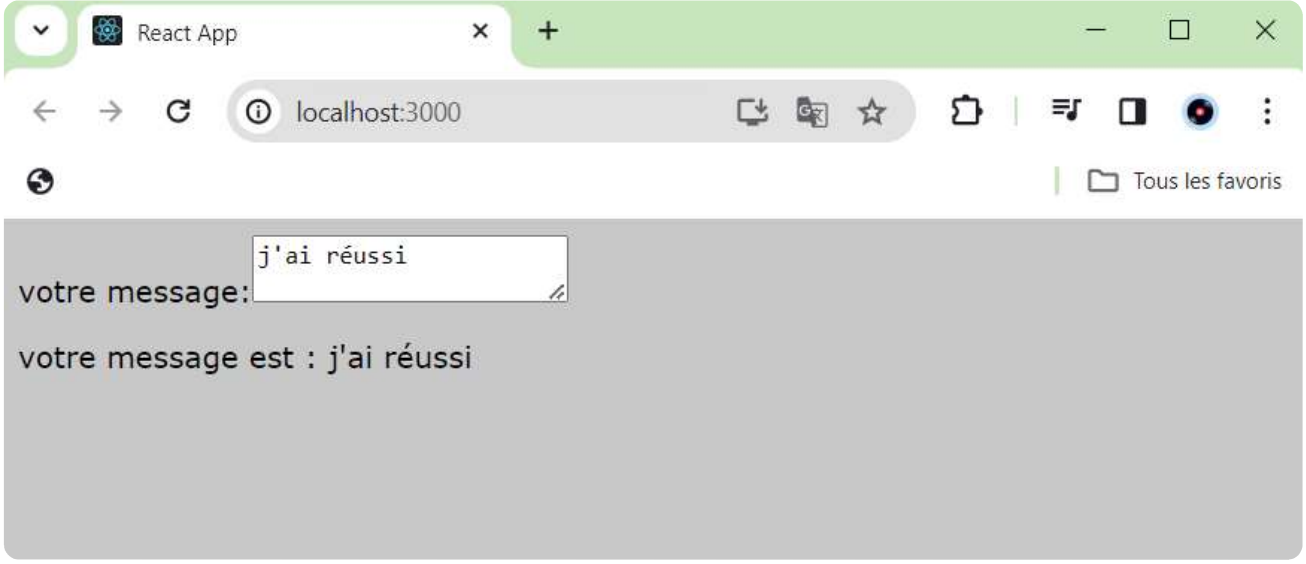
Le textarea a un **id**, un **name**, une valeur (value) liée à **formmsg**, et une fonction de rappel (onChange) pour appeler changer lorsque la valeur du champ de texte change.

`<div>...</div>`: En dehors du formulaire, un <**div**> est utilisé pour afficher le message saisi à côté du champ de texte.

{**formmsg**}: Cette expression est utilisée à l'intérieur du paragraphe pour afficher la valeur actuelle de formmsg. Voilà tout semble compréhensible.

Pour voir le résultat, taper la fameuse commande :

```
npm start
```

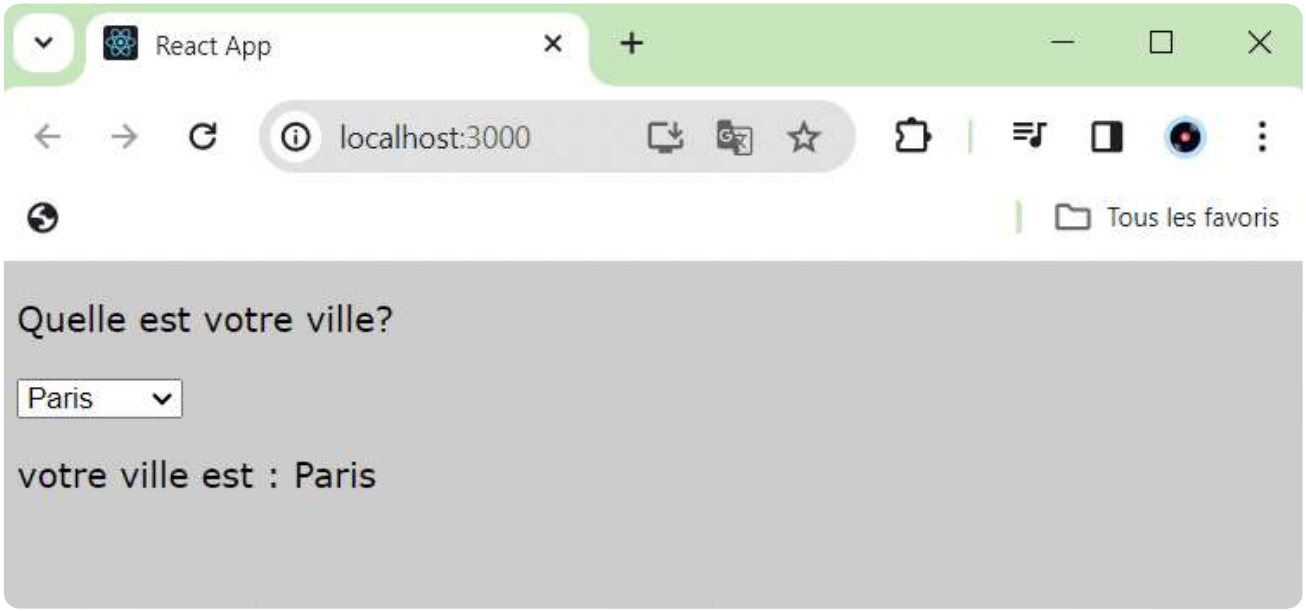


Voilà, il nous affiche bien notre message, parfait.

Contrôler un select

Dans cette section de cours, nous aborderons la manière de contrôler un élément `<select>` en utilisant React. Contrôler un `<select>` en React implique de gérer son état et ses options de manière à ce que les changements effectués par l'utilisateur soient pris en compte et reflétés dans l'interface utilisateur de manière cohérente.

Voilà un exemple de contrôle select que nous ressayons d'implémenter en reactjs :



Lorsque vous sélectionnez votre ville, cela entraîne instantanément l'affichage des informations correspondantes ci-dessous.

Pour ce faire, voyons la modification dans notre composant formulaire **: src/component/formulaire.js**

```
import { useState } from "react";
export default function Formulaire(){
const [formville,setFormville] = useState('Paris');
const changer = (e)=>{
setFormville(e.target.value);
}
return(
<>
<form  >
<p>Quelle est votre ville?</p>
<select
value={formville}
onChange={changer}
>
<option value="Paris">Paris</option>
<option value="Marseille">Marseille</option>
<option value="Paris">Paris</option>
</select>
</form>
<div>
<p>votre ville est : {formville}</p>
</div>
</>
)
}
```

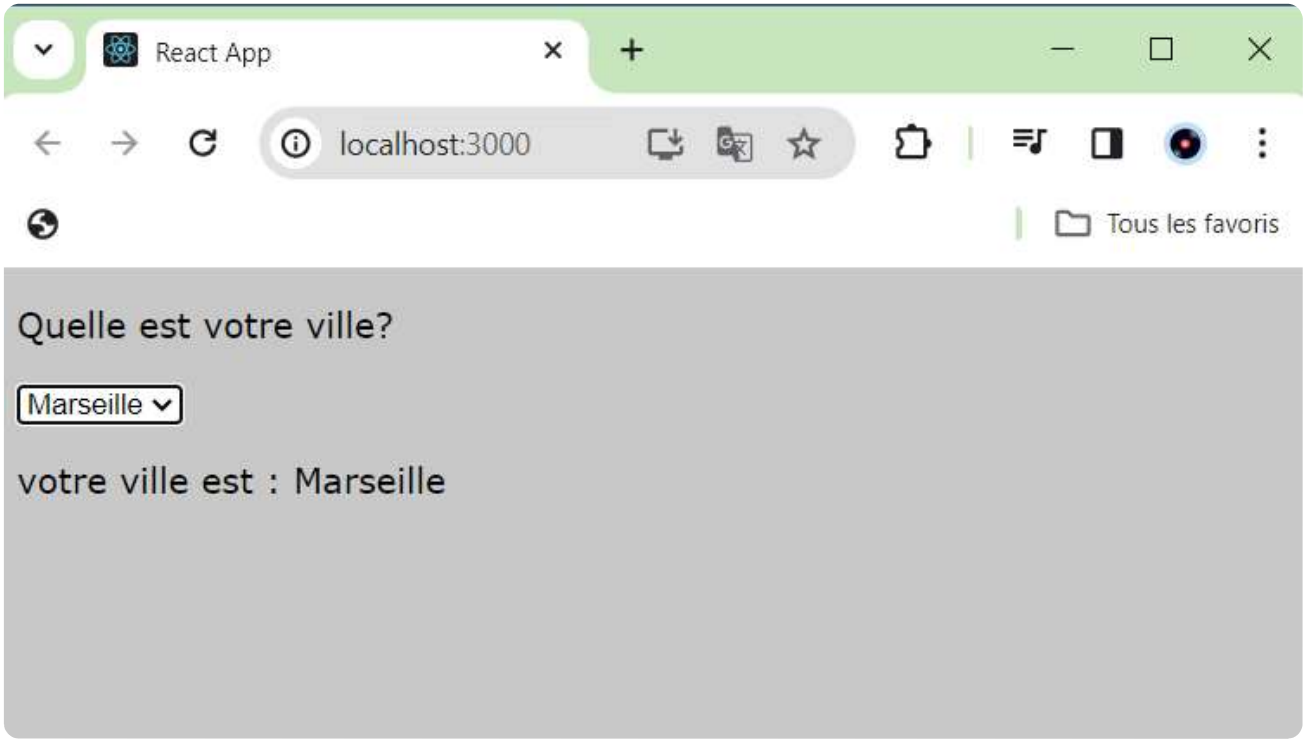
`const [formville, setFormville] = useState('Paris');:` Un état local est déclaré à l'aide de useState. **formville** est initialisé avec la valeur 'Paris' et **setFormville** est une fonction utilisée pour mettre à jour la valeur de **formville**.

Dans la partie de rendu `(return(...))` , un formulaire contenant un menu déroulant `(<select>)` est rendu.

Le `<select>` a une valeur (value) liée à **formville**, et une fonction de rappel (**onChange**) pour appeler changer lorsqu'une nouvelle option est sélectionnée.

Trois options sont définies dans le menu déroulant : Paris, Marseille et Paris.

Apres avoir lancer d'application, cela nous donne :



Excellent, nous parvenons à implémenter le contrôle du menu déroulant (select) en utilisant les fonctionnalités de ReactJS.